

Fiche de révision : Oral E6 · ColorRoom · Téo Tromprier (E2)

Format : 20 min présentation (PDF) → 20 min démonstration (app) → 20 min questions.

Objectif : connaître le projet ET le code. Tout ci-dessous est tiré du dépôt réel

NewGenesisAgency/color_room.

1. Pitch (30 secondes, à savoir par cœur)

La **ColorRoom** est un équipement scientifique du laboratoire **ENTPE / LTDS / BPMNP**, hébergé à **LUMEN – La Cité de la Lumière** (Lyon Confluence) : **2 cellules jumelles**, **42 plaques lumineuses** pilotées chacune par **32 canaux** couvrant tout le spectre visible. Le logiciel de recherche étant trop complexe pour l'initiation, j'ai développé une **application web de serious games**, embarquée sur **Raspberry Pi 5**, **100 % hors-ligne**. Ma partie (**E2**) couvre la base de données, l'API, l'interface, les jeux solo et multijoueur, la mesure colorimétrique et la documentation.

2. Les faits à connaître par cœur

Matériel (chiffres officiels : slide commune de l'équipe)

- **2 cellules jumelles** (2 salles d'analyse identiques) · **42 plaques** au total (**21 par cellule**).
- **Une plaque** : **80 cm × 80 cm** (≈ 23 cm d'épaisseur) · **2360 LED** · ≈ **300 W** rayonnés au maximum · LED disposées **en bandes sur les bords**.
- **32 canaux de commande = 32 types de LED** : **24 couleurs à spectre étroit** (du **proche UV** au **proche IR**) + **8 blancs** à différentes températures (et IR).
- **Liaison matérielle** : **bus série RS-485** (différentiel, multi-points, robuste sur longue distance) → le logiciel de supervision adresse chaque plaque / chaque canal sur ce bus.
- **Pont logiciel** : `supervision.exe` (sur le Pi) reçoit les commandes de mon API et les traduit en trames **RS-485** vers les plaques.

⚠ **À vérifier avant l'oral** : ma fiche disait avant « 18 spectrales + 14 phosphore ». La **slide officielle de l'équipe** dit **24 spectre étroit + 8 blancs = 32**. Retenir **24 + 8** (chiffre de l'équipe). Total = 32 canaux dans les deux cas.

Accès / réseau (à connaître pour la démo)

- **URL d'accès** : `http://172.17.40.39/` : depuis une **tablette** ou **n'importe quel appareil** connecté au Wi-Fi local **ColorRoom_WiFi**. (En interne le conteneur expose 3000, publié sur 8080 ; sur le réseau de la salle l'app répond à cette adresse.)
- Colorimètre CS-150/160 en **USB** sur le Pi · **Portainer** sur le port **9000**.

Pile logicielle (la mienne, E2 / équipe JavaScript)

- Next.js **16.2.1** (App Router), React **19.2.4**, TypeScript **5.5.3** (strict), better-sqlite3 **11.5.0**, Three.js **0.160.1**, Docker (multi-stage **arm64**), Portainer, Raspberry Pi **5**.

Équipe (8 : 2 sous-équipes)

- **Équipe 1 : JavaScript / React (E1 → E4)** : E1 Bonnevey (infra/Docker/CI-CD), **E2 Trompier = moi** (BDD, API, UI, jeux solo+IA+multi, mesure, doc), E3 Arbadji Ilyes (éditeur no-code + planification), E4 Akyuz (tests API).
- **Équipe 2 : Python / NiceGUI (E5 → E8)**.
- **Partenaires** : LUMEN (Cité de la Lumière, Lyon Confluence) · ENTPE / LTDS : labo **BPMNP** (Bio-ingénierie, Perception, Mécanique Numérique) · contacts Labayrade & Vella · prof Delbosc.

3. Démonstration minutée (20 min)

Prépare une **vidéo de secours** de chaque étape (au cas où le matériel/Pi bug).

1. **Connexion / inscription** (2 min) : créer un apprenant, montrer la détection de rôle, la connexion auto.
2. **Catalogue + un jeu solo** (4 min) : lancer **Color Speed** : montrer les dalles s'allumer en temps réel (3D + vraies dalles), le score.
3. **Puissance 4 contre l'IA** (4 min) : choisir un niveau (Novice → Légendaire), montrer que l'IA bloque une menace (anti-piège), parler du minimax.
4. **Multijoueur** (3 min) : Spectre Chromatique : 2 terminaux rejoignent, projection d'une couleur, scores.
5. **Mesure CS-160** (3 min) : connecter, mesurer une dalle, lire X Y Z / Lv / x,y, point sur le diagramme CIE.
6. **Tableau de bord enseignant** (2 min) : classes, scores, export CSV.
7. **Coulisses** (2 min) : Portainer (conteneurs), le `docker compose`, la base SQLite (fichier).

4. Le code à connaître (fichiers clés)

- `lib/db/index.ts` : connexion SQLite `singleton` (`dbSingleton`), `migrate()` : `PRAGMA journal_mode=WAL`, `synchronous=NORMAL`, `busy_timeout=5000`, `foreign_keys=ON` ; tables `crg_*` créées en `CREATE TABLE IF NOT EXISTS` + `ALTER` protégés ; `seedAdminFromEnv()` (compte admin via `.env`).
- `lib/auth.ts` : `hashPassword` / `verifyPassword` (**PBKDF2-HMAC-SHA512**, 100 000 itérations, sel 16 o, clé 64 o, format `sel:hash`) ; `createSession` (token `randomBytes(32)`, **30 jours**), `renewSession` (fenêtre glissante).
- `app/api/auth/register/route.ts` : inscription en **transaction atomique** (`db.transaction`).
- `app/api/auth/login/route.ts` : `verifyPassword` + cookie `crg_session` **HttpOnly + SameSite=lax** (30 j).
- `app/api/supervision/batch/route.ts` : proxy LED : **sémaphore** `HW_CONCURRENCY=2` + file d'attente (supervision.exe est quasi-série) + `AbortController` (timeout).
- `app/_components/GamePuissance4.tsx` : IA **minimax + élagage alpha-bêta**, `scoreWindow` (heuristique fenêtres de 4), profondeurs **1/2/5/9/12**, anti-piège (`winningCols`).
- `app/_components/Room3D.tsx` : vue 3D Three.js (WebGLRenderer, `forceContextLoss` au démontage, pause via Page Visibility API).
- `lib/multiplayer.ts` + `app/api/multiplayer/state/route.ts` : sessions persistées en base (`crg_mp_sessions.state_json`), lues en **polling**.

5 extraits de code montrés dans le deck (fichier + lien GitHub + lignes) :

- **Three.js** (slide 20) : `Room3D.tsx` : Scene + WebGLRenderer + boucle de rendu + `forceContextLoss` .

- **Variable transactionnelle ACID** (slide 22) : `db.transaction()` de `register/route.ts` .

- **Variable atomique · sémaphore** (slide 24) : `hwInFlight` de `supervision/batch/route.ts` .

- **Hachage PBKDF2** (slide 26) : `hashPassword` de `lib/auth.ts` .

- **Minimax (alpha-bêta)** (slide 34) : `scoreWindow` de `GamePuissance4.tsx` .

5. Fiche réponses du jury (Q&A)

Projet & cahier des charges

- « **À quoi sert la ColorRoom ?** » → Reproduire des éclairages à spectres précis pour la recherche sur la perception des couleurs ; mon appli la rend accessible en initiation via des jeux.
- « **Pourquoi des serious games ?** » → Le logiciel de recherche est trop technique ; le jeu rend la lumière/couleur compréhensible pour des apprenants.
- « **Qu'as-TU fait précisément ?** » → Partie E2 : BDD (7+ tables), API, UI/design, jeux solo + Puissance 4 (IA) + multijoueur, mesure CS-160, documentation. L'éditeur no-code est la partie E3.

Architecture & Next.js

- « **Pourquoi Next.js ?** » → Un seul projet front (React) + back (**Route Handlers**), un seul runtime Node à déployer sur le Pi ; **Server Components** pour le rendu, App Router.
- « **Architecture ?** » → Navigateur → routes API Next.js → couche `lib/` (auth, db, services) → SQLite + matériel (proxy supervision, pont CS-160). Tout conteneurisé.

Choix techniques (la question Node-RED)

- « **Pourquoi pas Node-RED comme proposé ?** » → Node-RED est **flow-based** : excellent pour automatiser des flux, mais inadapté à une vraie UI multi-pages (3D, catalogue, jeux), et ses flux JSON sont durs à versionner. React/TS/Next donnent des composants réutilisables, un **typage strict** (erreurs à la compilation), du code versionnable.
- « **Pourquoi TypeScript ?** » → Typage statique : un champ manquant ou une couleur mal formée est détecté par `tsc` /ESLint, pas en production.

Réseau / Docker / Raspberry Pi

- « **Comment déposes-tu ?** » → Image **Docker multi-stage** (deps → builder → runner) pour **arm64**, `docker compose up -d --build` , supervision par **Portainer**, app sur le port 8080.
- « **Et le réseau ?** » → Wi-Fi local **ColorRoom_WiFi** fourni par le Pi ; les tablettes/téléphones se connectent en local ; le colorimètre est en **USB** sur le Pi.
- « **Hors-ligne, vraiment ?** » → Oui : app + SQLite + matériel, tout local. (L'IA cloud Gemini est optionnelle ; un modèle local Ollama peut prendre le relais.)

Base de données / SQLite

- « **Pourquoi SQLite ?** » → Base **embarquée** (un simple fichier), zéro serveur à installer, idéale sur Pi ; via **better-sqlite3** (API **synchrone**, requêtes **préparées** = anti-injection).

- « **SQLite tient la charge ?** » → Pour quelques dizaines d'utilisateurs locaux, oui. **WAL** autorise des lectures concurrentes pendant une écriture, `busy_timeout` évite les erreurs `SQLITE_BUSY`.
- « **Migrations ?** » → `CREATE TABLE IF NOT EXISTS` + `ALTER` protégés : **idempotentes**, une base neuve se crée seule, une existante se met à jour sans casser.

Variables & persistance (questions « pointues »)

- **Atomique / transactionnelle** → `db.transaction()` (better-sqlite3) enveloppe `BEGIN/COMMIT/ROLLBACK` : l'inscription (user + adhésion classe) est **tout-ou-rien** (Atomicité ACID). Sinon : comptes « à moitié créés ».
- **Volatile (en mémoire)** → En JS il n'y a pas de mot-clé `volatile` ; je parle de l'**état runtime** stocké en `useRef` (ex. l'état des dalles pendant un jeu) : rapide, mais **non persistant**, perdu au rechargement : par opposition aux données **persistées** en SQLite.
- **Concurrente** → Le matériel (supervision.exe) est quasi-série ; un **sémaphore** `HW_CONCURRENCY=2` + file d'attente borne les appels simultanés.
- **Réactive** → `useState` : modifier la valeur déclenche le **re-rendu** ciblé de l'interface.
- **Environnement** → `process.env` (clé API, mot de passe admin) lue depuis `.env`, **hors Git**.
- **Persistante** → SQLite dans un **volume Docker** : survit aux redémarrages.

Sécurité / RGPD

- « **Comment stockes-tu les mots de passe ?** » → Jamais en clair : **PBKDF2-HMAC-SHA512**, 100 000 itérations, **sel aléatoire** 16 o, clé 64 o, format `sel:hash`. La vérification recalcule le hash et compare.
- « **Pourquoi PBKDF2 et pas bcrypt/argon2 ?** » → PBKDF2 est **natif** dans le module `crypto` de Node (aucune dépendance native à compiler sur arm64) ; 100 000 itérations donnent un coût correct. Argon2 serait plus moderne mais ajouterait une dépendance native.
- « **Les sessions ?** » → Token aléatoire (`randomBytes(32)`) en cookie **HttpOnly** (inaccessible au JS → anti-XSS) + **SameSite=lax** (anti-CSRF), **30 jours** avec **renouvellement glissant**.
- « **RGPD ?** » → Minimisation (un **pseudo** suffit, pas de nom/email) ; mots de passe hachés ; tout **local** (ENTPE) ; **droit à l'effacement** en cascade ; cloisonnement par rôle.

Jeux & temps réel

- « **Comment une dalle s'allume vite ?** » → Le navigateur ne parle pas directement au matériel : une **route proxy** relaie. Les **32 canaux** sont envoyés **en parallèle** (`Promise.all`, concurrence bornée) avec **timeout** (`AbortController`).
- « **La couleur à l'écran = la dalle ?** » → Oui, rendu **unifié** ; `CHANNEL_PROFILES` associe chaque canal à sa longueur d'onde réelle.

IA (Puissance 4)

- « **Explique ton IA.** » → **Minimax** : on simule l'arbre des coups, MAX pour l'IA / MIN pour l'adversaire ; **élagage alpha-bêta** coupe les branches inutiles ; **profondeur** limitée (1/2/5/9/12 selon le niveau). L'évaluation `scoreWindow` note chaque fenêtre de 4 cases : **défense pondérée plus fort que l'attaque** (-170 vs +130), victoire = `WIN_SCORE` (1 000 000), bonus de **centralité**. Anti-piège : l'IA ne donne pas une victoire immédiate à l'adversaire.
- « **Temps de réponse ?** » → Quasi instantané grâce à l'alpha-bêta et au poids central (exploration des bons coups en premier).

Multijoueur

- « **WebSockets ?** » → Non : l'état est **persisté en base** (`crg_mp_sessions.state_json`) et lu en **polling** de `/state`. Plus simple et robuste à déployer sur Pi, suffisant pour la cadence des jeux.

- « **Présence des joueurs ?** » → **Heartbeat** régulier ; jetons de session = **UUID** ; l'hôte (siège 1) démarre/arrête.

Mesure / colorimétrie

- « **Comment mesures-tu ?** » → Colorimètre **Konica Minolta CS-160** via un **pont .NET**. On lit le **tristimulus CIE XYZ**, la **luminance L_v (cd/m²)** et la **chromaticité (x, y)**, tracés sur le **diagramme CIE 1931**. Le CS-160 est **branché en USB** sur le Pi ; l'app l'appelle via une **API HTTP**.
- « **ΔE ?** » → Écart de couleur entre la mesure et la cible, utilisé pour scorer la précision dans les jeux de mesure.

Classes, QR code, multijoueur

- « **Comment un élève rejoint une classe ?** » → L'enseignant crée une classe et obtient un **code à 6 caractères** (ex. **CS5VHX**, sans 0/O ni 1/I) **et un QR code**. L'élève entre ce code à l'inscription (ou scanne le QR) → adhésion **atomique**.
- « **Et le code en multijoueur ?** » → Différent du code de classe : l'hôte d'une partie obtient un **code de salon** ; les autres le saisissent pour rejoindre la **session** (**crg_mp_sessions**), synchronisée en **polling**.

Méthode & gestion de projet

- « **Quelle méthode ?** » → **Agile** : développement **incrémental**, livraisons régulières, ajustements. **Échanges réguliers avec le client M. Labayrade** (directeur du labo **BPMNP**).
- « **Gestion du code ?** » → **Git/GitHub** : je développe sur la branche **ux-last** (intégration), puis je **merge** sur **main** (stable) ; ~280 commits clairs, **CI/CD** GitLab → build Docker → Pi.
- « **La documentation ?** » → **C'est moi qui l'ai rédigée** : guide technique, **15 diagrammes UML**, notice apprenant (l'aide), manuel d'installation (README).

Qualité / tests

- « **Tests ?** » → Vérification de **types (tsc)** + **build de production** à chaque étape ; tests de l'API et fiches de recette côté E4 ; transactions ACID, exports UTF-8 + BOM.

UML

- « **Différence ERD / diagramme de classes ?** » → L'**ERD** modélise les **tables relationnelles** (clés primaires/étrangères) ; le **diagramme de classes** modélise la **conception objet** (TypeScript).

6. Questions pièges : réponses courtes

- « **C'est toi qui as tout fait ?** » → Non, projet d'équipe ; je présente **ma partie E2** (le cœur de l'appli), qui s'appuie sur l'infra de E1 et est testée par E4.
- « **Et si une dalle tombe en panne ?** » → L'API gère par plaque, l'appli continue, le jeu ne plante pas.
- « **Combien de joueurs simultanés ?** » → 1 dalle/joueur (jusqu'à 42) en local, limité par le Wi-Fi de la salle.
- « **Pourquoi pas une vraie base serveur (PostgreSQL) ?** » → Surdimensionné pour un déploiement embarqué mono-poste hors-ligne ; SQLite suffit et simplifie l'install.
- « **L'IA générative dans le projet ?** » → Hors dossier E2 ; c'est une **exploration personnelle** (génération de jeux assistée par IA locale Ollama) que je peux montrer en bonus.

7. Script slide par slide : 49 slides (≈18 min parlé + vidéo 2 min)

Deck = [ColorRoom_Presentation.pdf/.pptx](#) (49 slides). ⌚ = durée cible. Parlé ≈ 18 min + vidéo

(slide 49) 2 min = 20 min. ★ = ne jamais sacrifier · ⚡ = montrer vite si en retard.

Bloc A : contexte, équipement, équipe, gestion (1 → 12) · ~4:30

#	Slide	⌚	Ce que je dis
1	Titre	0:15	« Téo Tromprier, candidat E2 : projet ColorRoom – Serious Games pour LUMEN / ENTPE . »
2	Sommaire	0:15	Annoncer le plan.
3	Contexte & partenaire	0:35	ENTPE (École Nat. des Travaux Publics de l'État) / LTDS , équipe BPMNP (Bio-ingénierie, Perception, Mécanique Numérique), à LUMEN (Cité de la Lumière, Lyon Confluence).
4	La ColorRoom (photo réelle)	0:40	2 cellules jumelles ; chaque cellule tapissée de plaques sur les murs ET le plafond ; 42 plaques (21/cellule).
5	La plaque lumineuse	0:40	80 × 80 cm (ép. ~23 cm), 2360 LED , ~300 W, 32 canaux = 24 spectres étroits (proche UV → IR) + 8 blancs , bus RS-485 .
6	Appli de supervision actuelle	0:30	Logiciel réservé aux chercheurs, trop complexe (grille + 32 sliders + spectre) → besoin d'un outil pédagogique .
7	Le besoin	0:30	Rendre la salle accessible par des serious games ; objectifs / contraintes (Pi, hors-ligne, latence).
8	Cas d'utilisation (plein écran)	0:35	Acteurs Apprenant / Enseignant , include/extend, légende couleur = responsable (E1 → E4).
9	Équipe	0:25	8 étudiants, 2 sous-équipes : JS/React (E1 → E4, dont moi E2) + Python/NiceGUI (E5 → E8).
10 ⚡	Gantt (plein écran)	0:20	Projet planifié , ma contribution (E2) suivie tout du long.
11 ★	Gestion de projet	0:40	Agile + Kanban (À faire/En cours/Terminé) ; échanges client M. Labayrade (directeur BPMNP) ; Git ux-last → merge main ; CI/CD ; docs rédigées par moi .
12	Transition · Ma contribution E2	0:10	« Place à ma partie : architecture, données, sécurité, jeux, IA, mesure. »

Bloc B : architecture & choix (13 → 18) · ~3:30

#	Slide	🕒	Ce que je dis
13	Architecture (composants)	0:35	Route Handlers → lib/ → SQLite + matériel. *(le Pi est sur le déploiement)*.
14 ⚡	Diagramme de classes	0:25	Entités typées TypeScript .
15 ★	Choix techniques (comparatif)	1:00	Node-RED flow-based inadapté / React (Virtual DOM), TS strict , Next.js (un seul runtime).
16	Pile technique	0:25	Next 16, React 19, TS 5.5, better-sqlite3, Three.js, Docker arm64.
17	Réseau & déploiement	0:45	Pi 5 + Docker ; dalles RS-485, CS-160 USB, API App ↔ CS-160 ; cardinalités.
18	Configuration	0:25	URL d'API à chaud , test /health , banc des 32 canaux .

Bloc C : ma partie E2 (19 → 41) · ~9:00

#	Slide	🕒	Ce que je dis
19	Interface	0:30	UI 3D temps réel (Three.js), pages, menu par rôle.
20	Code · Three.js	0:35	Scene + WebGLRenderer , un Mesh/dalle, boucle <code>requestAnimationFrame</code> , <code>forceContextLoss</code> .
21	Base de données (ERD)	0:30	better-sqlite3, WAL , migrations idempotentes, FK.
22 ★	Code · transactionnelle (ACID)	0:40	<code>db.transaction()</code> : user + classe, tout-ou-rien (ROLLBACK).
23	Typologie des variables	0:35	transactionnelle, volatile (<code>useRef</code>), persistante, concurrente, réactive, environnement.
24 ★	Code · atomique (sémaphore)	0:40	<code>hwInFlight</code> borné à 2 , file de Promises, mono-thread .
25	Sécurité	0:30	Cookie HttpOnly+SameSite , RGPD (pseudo).
26	Code · PBKDF2	0:30	<code>pbkdf2Sync(..., 100_000, 64, 'sha512')</code> , <code>sel:hash</code> .
27 ⚡	Séquence · connexion	0:18	<code>verifyPassword</code> → session (30 j) → cookie.
28	Tableau de bord enseignant	0:35	Classes : code 6 car. (<code>CS5VHX</code>) + QR code ; suivi élèves ; gestion utilisateurs ; CSV .
29	Parcours apprenant	0:30	Inscription 3 étapes ; rejoindre une classe via le code ; atomique + connexion auto.
30	Jeux solo	0:30	Color Speed, Simon, Tetris, P4 ; 32 canaux en parallèle.
31 ⚡	Séquence · exécution jeu	0:18	Clic → Runtime → <code>/api/supervision/batch</code> → dalles.
32 ⚡	États · partie	0:12	Prête → en cours → terminée.
33 ★	IA Puissance 4	0:50	Minimax + alpha-bêta , profondeurs 1/2/5/9/12, défense > attaque , anti-piège. *(pas un réseau de neurones)*
34 ★	Code · minimax	0:35	<code>scoreWindow</code> + <code>if (alpha >= beta) break;</code> .
35	Jeux multijoueur	0:30	Sans WebSocket : état persisté lu en polling ; code de salon , heartbeat.
36 ⚡	Séquence · multijoueur	0:18	Hôte crée → invité saisit le code → synchro.
37	Éditeur + IA générative	0:30	Éditeur no-code (E3) ; mon exploration « Créer avec l'IA » (Ollama local).
38	Physique de la lumière	0:35	CS-160 : XYZ , Lv , x,y , CIE 1931 , ΔE *(défs sous la slide)*.
39	Séquence · mesure CS-160	0:25	App → <code>/api/cs160</code> → pont .NET → CS-160 (USB) → XYZ → CIE → ΔE.
40 ⚡	États · CS-160	0:12	Déconnecté → connecté → mesure → résultat.

#	Slide	🕒	Ce que je dis
41 ⚡	Activité · remap	0:12	RGB → 32 canaux .

Bloc D : aide, qualité, clôture + vidéo (42 → 49) · ~2:30 (+2 min vidéo)

#	Slide	🕒	Ce que je dis
42 ⚡	Aide intégrée	0:18	Page laide hors-ligne.
43	Qualité & doc	0:25	tsc + build prod ; ACID ; tests API E4 ; 15 diagrammes UML (par moi).
44	Difficultés / solutions	0:30	CORS & pare-feu (→ suivante), latence, SQLITE_BUSY (WAL), multi (polling).
45	Notes réseau Windows	0:25	CORS : New-NetFirewallRule + portproxy 18080 → 8080.
46	Conclusion / bilan	0:30	« Cœur applicatif livré : données, API, UI, jeux, IA, multi, mesure. Embarqué, hors-ligne, robuste. »
47	Place à la démonstration	0:10	« Place à la démonstration. »
48	Merci / questions	0:05	(transition)
49 ★	Vidéo du projet	2:00	Lancer la vidéo (la ColorRoom réelle) → enchaîne sur la démo live.

Total ≈ 18 min parlé + 2 min vidéo = 20:00. En retard → couper les « ⚡ ». Ne jamais sacrifier les ★ (11, 15, 22, 24, 33, 34, 49).

8. Mémo « infos complexes que je risque d'oublier »

- **ColorRoom** : 2 cellules jumelles ; plaques sur murs + plafond ; 42 plaques (21/cellule).
- **Plaque** : 80×80 cm (ép. ~23 cm), 2360 LED, ~300 W, 32 canaux = 24 spectres étroits + 8 blancs, bus RS-485.
- **Sigles** : ENTPE = École Nationale des Travaux Publics de l'État · LTDS = Lab. de Tribologie et Dynamique des Systèmes · BPMNP = Bio-ingénierie, Perception, Mécanique Numérique · LUMEN = Cité de la Lumière.
- **Existant** : appli de supervision **trop complexe** (réservée aux chercheurs) → d'où ColorRoomGames.
- **Méthode** : agile + Kanban ; client M. Labayrade (directeur BPMNP) ; moi = E2 ; docs rédigées par moi.
- **Git** : branche **ux-last** (intégration) → merge **main** (stable) ; CI/CD GitLab → Docker → Pi.
- **Code de classe** (**CS5VHX** + QR code) pour rejoindre une classe ≠ code de salon multijoueur (rejoindre une partie).
- **CS-160** : **USB** sur le Pi ; appelé via **API HTTP** ; XYZ / Lv / x,y / **CIE 1931** / ΔE.
- **RS-485** : bus série multi-points pour les 42 plaques. **WAL** : lectures concurrentes. **PBKDF2** : 100 000 itér. + sel. **Alpha-bêta** : coupe les branches inutiles. **Sémaphore** **HW_CONCURRENCY=2** .
- ⚠ **IA du Puissance 4** = minimax, **PAS** un réseau de neurones.
- **URL démo** : <http://172.17.40.39/> (Wi-Fi ColorRoom_WiFi).

9. La vidéo de 2 min : slide-pont (slide 49)

Vidéo **embarquée** dans le **.pptx** (dernière slide) = **pont présentation** → **démonstration**. Je la lance à la fin du parlé, puis j'enchaîne sur la démo live.

1. La vraie **ColorRoom** est à **LUMEN**, pas dans la salle : seul moyen de montrer le **système réel complet**.
2. Elle **clôt** la présentation et **ouvre** la démo.
3. **Filet de sécurité** si le matériel bug.

 Lecture : clic sur la vidéo (ou auto en diaporama). Garder **video_demo.mp4** à côté en secours.

Bon courage Téo : section 5 = 90 % des questions, section 7 = ton minutage (49 slides, 20 min).